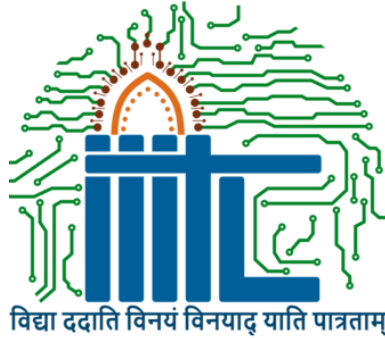


INDIAN INSTITUTE OF INFORMATION TECHNOLOGY LUCKNOW

M.Tech (Computer Science)



Project Report

on

NLP-Based CVE Threat Intelligence Dashboard

Submitted By:

- | | | | |
|-----------------|------------|------------------|------------|
| 1. Shivani M | (MCS25008) | 5. Aryan Verma | (MCS25041) |
| 2. Tejeswani | (MCS25020) | 6. Madhu Shalini | (MCS25043) |
| 3. Aarohi Kumar | (MCS25022) | 7. Rose Gupta | (MCS25045) |
| 4. Karunakar | (MCS25039) | | |

Under the Supervision of
Dr. Gaurav Sharma

Abstract

Software vulnerabilities disclosed through Common Vulnerabilities and Exposures (CVE) records grow in volume every year, and each disclosure typically arrives as unstructured free-text alongside sparse metadata. For security teams, manually reading, scoring, and categorizing this steadily increasing stream of disclosures has become impractical, creating a clear need for automation that can keep pace with the rate of new threats. This project presents the CVE Threat Intelligence Dashboard, an enterprise-grade, AI-powered cybersecurity analytics platform designed to analyze, classify, predict, and visualize software vulnerabilities in real time using a combination of Natural Language Processing, classical Machine Learning, and probabilistic sequence modeling.

Rather than relying on a single predictive model, the system deliberately layers several complementary techniques so that their outputs can cross-validate one another. A Random Forest regressor, trained on TF-IDF vectors of unigrams and bigrams extracted from CVE descriptions, predicts numerical CVSS scores and maps them to categorical severity levels of Low, Medium, High, or Critical. In parallel, a Gaussian Hidden Markov Model and a Maximum Entropy Markov Model analyze the sequential structure of vulnerability text to detect hidden threat states and model how threats transition between them over time. To capture deeper semantic meaning, the system also incorporates transformer-based deep learning: a DistilBERT model fine-tuned for multi-class sequence classification categorizes vulnerabilities into types such as SQL Injection, Buffer Overflow, Cross-Site Scripting, Remote Code Execution, and Denial of Service, while a zero-shot BART classifier, built on a Natural Language Inference pipeline, provides flexible severity classification on arbitrary text without requiring additional task-specific training.

The predictions generated by these five models are aggregated into a single, unified threat-intelligence output and persisted in a local SQLite-backed vulnerability vault, which supports historical tracking, severity-based filtering, and CSV export for downstream reporting. The entire pipeline is served through a FastAPI backend and exposed through an interactive, glassmorphism-styled web dashboard organized into distinct modules: a Vulnerability Center for query-level analysis, a Trend Analytics view for exploit-vector and component-level breakdowns, a CVSS Distribution dashboard for statistical severity analysis, a Time-Series Forecast module powered by ARIMA and linear regression that projects vulnerability trends from 2017 through 2028, a Markov Threat Sequences view for visualizing HMM-derived threat states, and a Model Engine Configuration panel for retraining and fine-tuning the underlying models directly from the interface.

Overall, the project shows how classical machine learning, sequence modeling, and transformer-based NLP can be combined within a single, cohesive architecture to transform raw, unstructured CVE text into structured, real-time, and interpretable threat intelligence, offering a practical foundation for enterprise-grade vulnerability management.

Contents

Abstract	i
1 Introduction	1
1.1 Problem Statement	1
1.2 Objectives	1
2 Repository Overview	2
2.1 Technology Stack	2
2.2 Machine Learning Models	2
3 System Architecture	3
3.1 Architecture Diagram	3
3.2 Architecture Overview	4
3.3 Frontend	4
3.4 Backend	4
3.5 Machine Learning Engine	4
4 Methodology	5
4.1 Workflow Diagram	5
4.2 Data Collection	5
4.3 Data Preprocessing	6
4.4 Feature Extraction	6
4.5 Machine Learning Analysis	6
4.6 Threat Classification	6
4.7 Visualization and Analytics	6
4.8 Database Storage	7
5 Machine Learning Pipeline	8
5.1 Random Forest (CVSS & Severity Predictor)	8
5.2 Gaussian HMM (Sequential Clustering)	8
5.3 MEMM (Vulnerability Sequence Tagger)	9

5.4	Fine-Tuned DistilBERT (Vulnerability Categorizer)	9
5.5	Zero-Shot BART (Dynamic Severity Classifier)	10
5.6	Combined Pipeline Behavior	11
6	Implementation	12
6.1	Development Environment	12
6.2	Installation and Execution	12
6.3	Backend Implementation	13
6.4	System Integration	13
7	Dashboard Modules and User Interface	14
7.1	Vulnerability Center	14
7.2	Trend Analytics	14
7.3	CVSS Distribution Dashboard	15
7.4	Time-Series Forecast	15
7.5	Markov Threat Sequences	16
7.6	SQLite Vulnerability Vault	16
7.7	Model Engine Configuration	17
8	Advantages and Limitations	18
8.1	Advantages	18
8.2	Limitations	18
9	Conclusion and Future Scope	19
9.1	Conclusion	19
9.2	Future Scope	19

List of Figures

3.1	Architecture of the CVE Threat Intelligence Dashboard	3
4.1	Workflow of the CVE Threat Intelligence Dashboard	5
7.1	Web dashboard for manual email analysis and real-time prediction	14
7.2	Web dashboard for manual email analysis and real-time prediction	15
7.3	Web dashboard for manual email analysis and real-time prediction	15
7.4	Web dashboard for manual email analysis and real-time prediction	16
7.5	Web dashboard for manual email analysis and real-time prediction	16
7.6	Web dashboard for manual email analysis and real-time prediction	17
7.7	Web dashboard for manual email analysis and real-time prediction	17

1 Introduction

Every year, thousands of new software vulnerabilities are disclosed through the Common Vulnerabilities and Exposures (CVE) system. Each CVE entry typically consists of a free-text description along with structured metadata such as its identifier, affected component, and (where available) a CVSS score. For security teams, the sheer volume of new disclosures makes manual review a significant bottleneck – vulnerabilities need to be triaged, scored, categorized, and prioritized quickly to keep systems safe.

Natural Language Processing offers a way to automate much of this triage. By treating each CVE description as a text-classification and regression problem, models can be trained to predict severity, identify the underlying vulnerability class (e.g. SQL Injection, Buffer Overflow, RCE), and detect patterns in how threats evolve over time.

This project was designed to build a complete threat-intelligence pipeline around this idea: rather than relying on a single model, it combines statistical machine learning, sequence modeling, and transformer-based deep learning into one dashboard, so that different techniques can cross-validate and complement each other's predictions.

1.1 Problem Statement

The problem is to automatically analyze raw CVE descriptions and produce structured, actionable threat intelligence. The system should be able to:

- ingest free-text CVE descriptions and associated metadata,
- predict a numerical CVSS severity score and a categorical severity level,
- classify the vulnerability into a specific exploit category,
- model how threats transition between states over time,
- forecast future vulnerability trends,
- store and visualize all analyzed results for further querying.

1.2 Objectives

The project aims to:

- automate CVE vulnerability analysis end-to-end,
- classify cyber threats using NLP,
- predict vulnerability severity scores,
- visualize exploit trends and attack vectors,
- implement HMM and MEMM sequence modeling for threat-state analysis,
- forecast future vulnerability trends using time-series methods.

2 Repository Overview

The repository implements a complete AI-driven threat-intelligence platform. It includes a FastAPI backend, an interactive analytics dashboard, a multi-model machine learning engine, and an SQLite-backed vulnerability vault.

2.1 Technology Stack

Technology	Purpose
Python	Core language for backend, ML pipeline, and utilities.
FastAPI	REST API backend and application server.
SQLite	Persistent storage for the vulnerability vault.
HTML5 / CSS3	Glassmorphism-styled dashboard frontend.
JavaScript	Dynamic dashboard interactions and visualizations.
Transformers (Hugging-Face)	DistilBERT and BART model integration.
PyTorch	Deep learning inference and fine-tuning.
Scikit-learn	Random Forest, TF-IDF, and classical ML utilities.
Git & GitHub	Version control.

Table 2.1: Software stack used in the project

2.2 Machine Learning Models

Model	Purpose
Random Forest Regressor	CVSS score prediction
Gaussian Hidden Markov Model	Threat sequence clustering
MEMM	Threat transition prediction
DistilBERT	Vulnerability categorization
Zero-Shot BART	Severity classification
ARIMA	Time-series forecasting

Table 2.2: Machine learning models used in the system

3 System Architecture

The system follows a modular client-server architecture, with a clear separation between the presentation layer, the API layer, and the machine learning engine.

3.1 Architecture Diagram

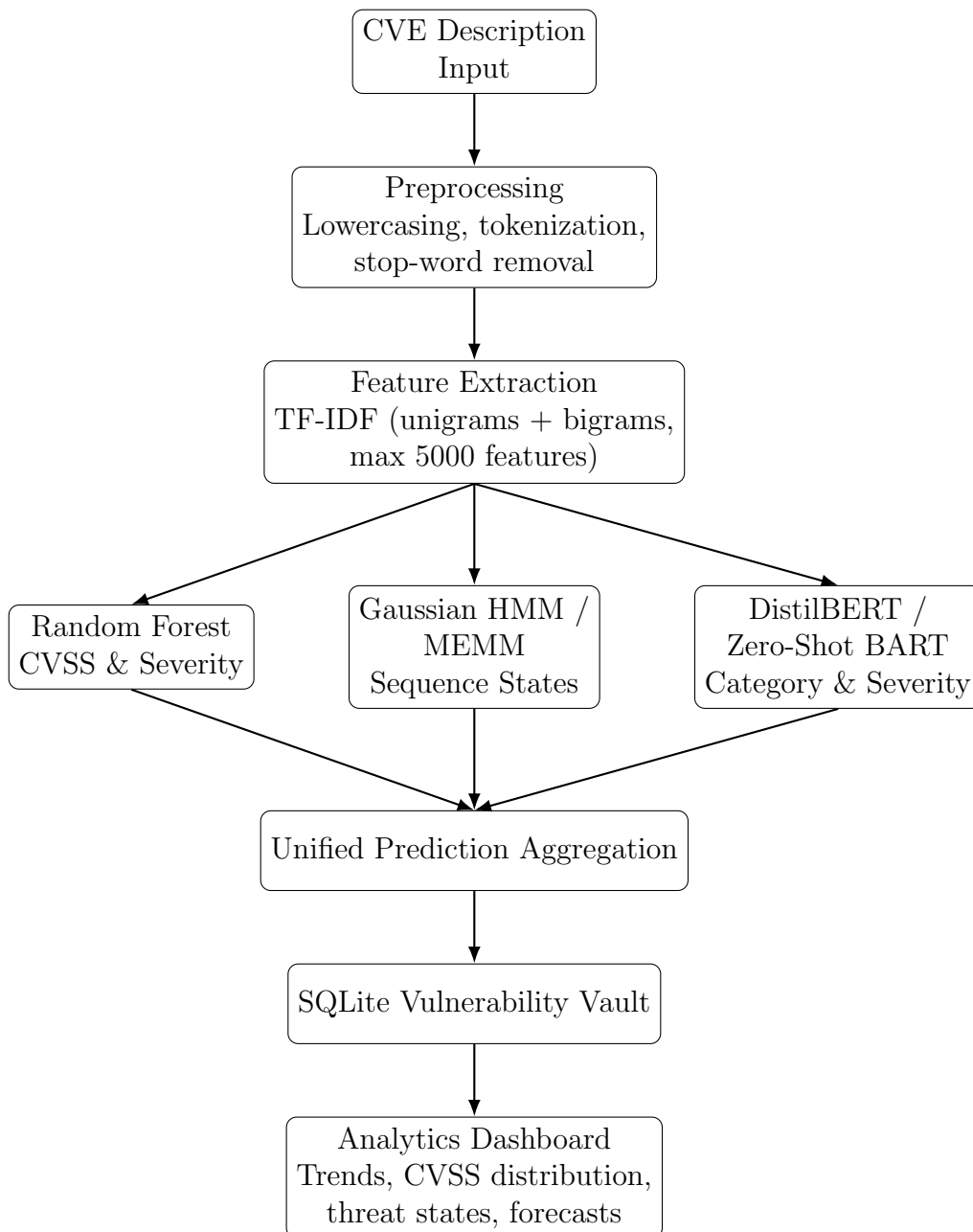


Figure 3.1: Architecture of the CVE Threat Intelligence Dashboard

3.2 Architecture Overview

The system is composed of the following main components:

- Input Layer
- Preprocessing Layer
- Feature Extraction Layer
- Machine Learning Engine (parallel model inference)
- Prediction Aggregation Layer
- Storage Layer (SQLite Vulnerability Vault)
- Visualization / Dashboard Layer

3.3 Frontend

- Interactive glassmorphism-styled dashboard
- Real-time analytics visualization
- Trend monitoring interface
- CVE search and filtering

3.4 Backend

- FastAPI server exposing REST endpoints
- SQLite database integration
- Machine learning inference engine
- Model retraining and configuration endpoints

3.5 Machine Learning Engine

- TF-IDF vectorization of CVE text
- Transformer model inference (DistilBERT, BART)
- HMM / MEMM sequence decoding
- ARIMA-based forecast analytics

4 Methodology

The CVE Threat Intelligence Dashboard follows a multi-stage AI-driven pipeline for processing vulnerability descriptions and generating threat intelligence predictions.

4.1 Workflow Diagram

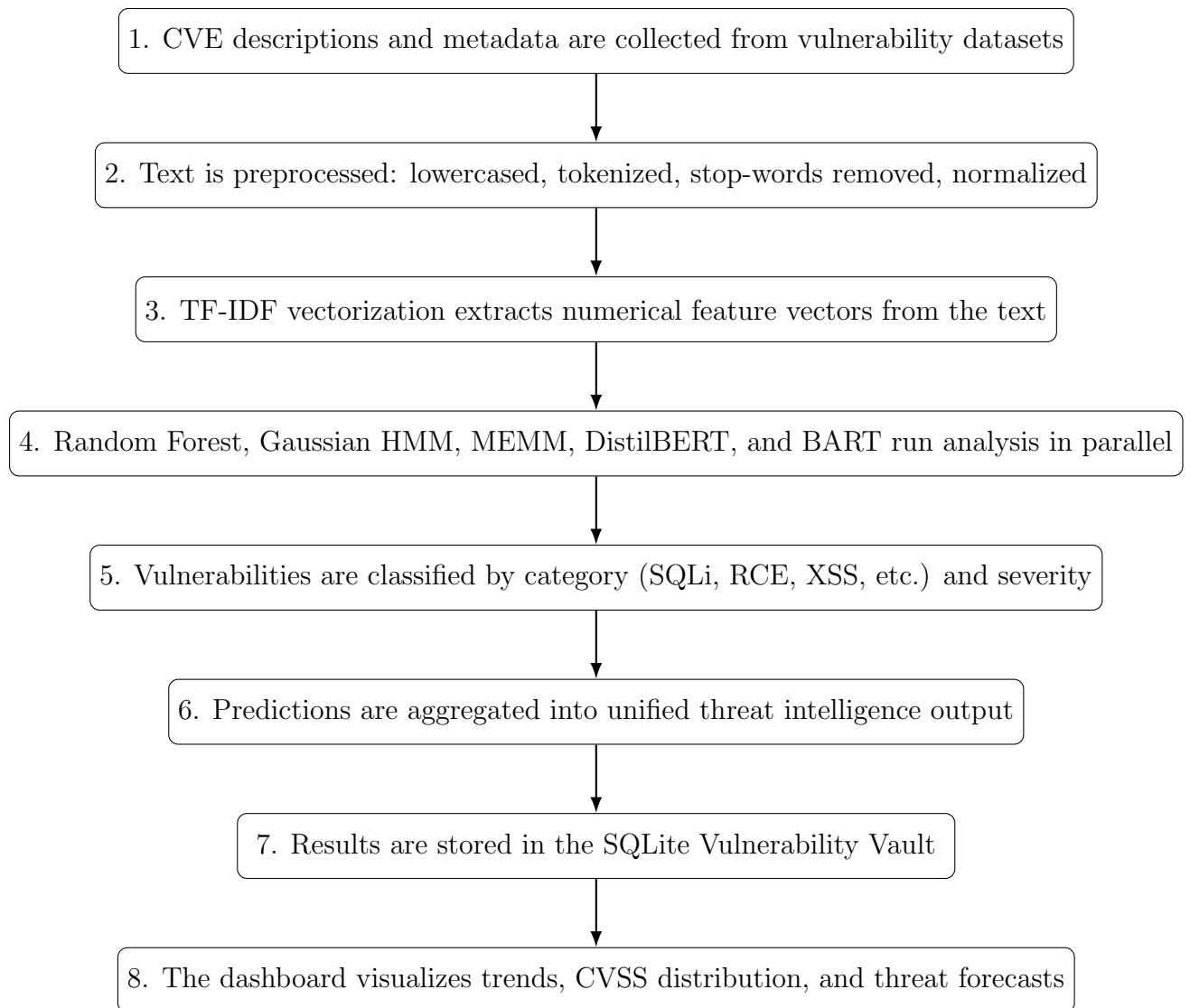


Figure 4.1: Workflow of the CVE Threat Intelligence Dashboard

4.2 Data Collection

The system collects vulnerability-related information from CVE descriptions and cybersecurity datasets, including CVE identifiers, vulnerability descriptions, CVSS scores,

severity levels, and exploit categories. This data is used to train and evaluate the machine learning and NLP models.

4.3 Data Preprocessing

Textual vulnerability descriptions are preprocessed using standard NLP techniques: lowercase conversion, tokenization, stop-word removal, TF-IDF vectorization, and text normalization. The processed data is transformed into numerical feature vectors suitable for the machine learning models.

4.4 Feature Extraction

TF-IDF vectorization extracts meaningful textual features from CVE descriptions, helping to identify important vulnerability keywords, improve classification accuracy, and reduce dimensional complexity. The generated vectors feed into the Random Forest, HMM, and MEMM models.

4.5 Machine Learning Analysis

The processed vulnerability vectors are analyzed using multiple models:

- The Random Forest Regressor predicts CVSS severity scores.
- The Gaussian Hidden Markov Model detects hidden threat states.
- The MEMM predicts vulnerability transition sequences.
- DistilBERT classifies vulnerability categories.
- Zero-Shot BART performs semantic severity classification.

The outputs of all models are combined to generate unified threat intelligence predictions.

4.6 Threat Classification

Vulnerabilities are classified into categories such as SQL Injection, Buffer Overflow, Cross-Site Scripting (XSS), Remote Code Execution (RCE), and Denial of Service (DoS). Severity levels are classified as Low, Medium, High, or Critical.

4.7 Visualization and Analytics

Generated predictions are visualized on interactive dashboards showing CVSS score distributions, threat trends, exploit vector analytics, time-series forecasts, Hidden Markov threat states, and vulnerability history tracking.

4.8 Database Storage

Analyzed vulnerability information is stored in an SQLite database, maintaining CVE IDs, prediction results, severity levels, exploit vectors, and historical analytics logs. This allows future querying, filtering, and export.

5 Machine Learning Pipeline

This chapter describes each model in the pipeline in detail, covering its role in the system, its input representation, its internal architecture and training procedure, and its practical strengths and limitations. The five models operate largely independently at inference time and are combined only at the aggregation stage, which allows each one to be retrained, replaced, or tuned without affecting the others.

5.1 Random Forest (CVSS & Severity Predictor)

Goal: Predict a numerical CVSS score in the range 0–10 and map it to a categorical severity level of Low, Medium, High, or Critical.

Input Representation: The cleaned vulnerability description is converted into a TF-IDF matrix using unigrams and bigrams, capped at a maximum of 5,000 features to keep the feature space computationally manageable while still capturing multi-word phishing-relevant phrases such as “remote code execution” or “buffer overflow.”

Algorithm and Hyperparameters: The regressor is a scikit-learn `RandomForestRegressor` configured with 100 estimators and a maximum tree depth of 15. Bagging across 100 trees reduces variance compared to a single decision tree, while the depth cap prevents individual trees from overfitting to rare vocabulary in the training corpus.

Training Process: The dataset of labeled CVE descriptions and their known CVSS scores is split into training and validation sets. The TF-IDF vectorizer is fit on the training split only, and the resulting feature matrix is used to train the regressor. Predicted continuous scores are then bucketed into the four severity classes using fixed CVSS threshold boundaries.

Strengths: Fast to train and to run inference with, resistant to overfitting due to ensembling, and produces feature-importance scores that make the model’s reasoning partially interpretable.

Limitations: As a bag-of-words-style model, it cannot capture word order or long-range context, so it can struggle with descriptions where severity depends on subtle phrasing rather than the presence of specific keywords.

5.2 Gaussian HMM (Sequential Clustering)

Goal: Group sequences of vulnerability descriptions into a small number of latent threat states and detect transitions between them over time, effectively treating the vulnerability stream as a time-ordered sequence rather than a set of independent records.

Input Representation: Continuous-valued TF-IDF feature matrices (rather than discrete tokens) are fed directly into the model, since a Gaussian HMM assumes real-valued, normally distributed emissions at each hidden state.

Algorithm: A Gaussian Hidden Markov Model is fit to the sequence of feature vectors. The model learns a fixed number of hidden states (e.g. Low Threat, Medium Threat, High Threat), a transition-probability matrix describing how likely the system is to move from one state to another, and a Gaussian emission distribution describing the expected feature vector for each state.

Logic: At inference time, the Viterbi algorithm (or an equivalent forward-backward procedure) is used to decode the most probable hidden-state sequence for a batch of vulnerabilities, revealing whether the overall threat landscape is trending toward a calmer or more volatile state.

Role in the Ensemble: Unlike the Random Forest and transformer models, which score individual CVE entries in isolation, the HMM contributes a *temporal* signal – it helps answer “is the threat landscape as a whole escalating?” rather than “how severe is this one CVE?”

5.3 MEMM (Vulnerability Sequence Tagger)

Goal: Predict transitions between vulnerability tags and structural threat signatures, complementing the HMM’s unsupervised state discovery with a supervised, feature-rich transition model.

Algorithm: A Maximum Entropy Markov Model is implemented as a logistic regression classifier that is conditioned on transition-history features – that is, in addition to the current observation’s own features, the model also uses the previously predicted tag as an input feature when predicting the next one.

Why MEMM over HMM here: Where the Gaussian HMM assumes emissions depend only on the current hidden state, the MEMM can incorporate arbitrary, overlapping features of the observation (such as TF-IDF weights, keyword flags, or metadata) alongside the tag history, which typically improves tagging accuracy at the cost of losing the HMM’s fully generative probabilistic guarantees.

Output: A predicted sequence of structural threat tags across a batch of vulnerabilities, used alongside the HMM’s state sequence to characterize how threats evolve.

5.4 Fine-Tuned DistilBERT (Vulnerability Categorizer)

Goal: Predict the specific vulnerability category (e.g. SQL Injection, Buffer Overflow, Cross-Site Scripting, Remote Code Execution, Denial of Service) directly from the raw, unprocessed CVE description text.

Architecture: The model is built on `distilbert-base-uncased`, a distilled version of BERT with roughly 40% fewer parameters and about 60% faster inference than full BERT, while retaining approximately 97% of its language-understanding performance. A classification head (a linear layer over the pooled [CLS] token representation) is attached on top, configured for 4 output classes.

Tokenization and Input: CVE descriptions are tokenized using DistilBERT’s Word-Piece tokenizer, truncated or padded to a fixed maximum sequence length, and passed through the transformer to obtain contextual embeddings.

Training Details: The classification head (and optionally the underlying transformer weights) is fine-tuned on the `ag_news` text-classification dataset for sequence classification, run for 2 epochs with a learning-rate scheduler that decays the rate over the course of training to stabilize convergence. Cross-entropy loss is used as the training objective.

Strengths: Because it is transformer-based, DistilBERT captures word order and contextual meaning far better than the TF-IDF-based Random Forest, allowing it to distinguish between superficially similar descriptions that describe different vulnerability classes.

Limitations: Requires more compute and memory than the classical models, and fine-tuning quality is sensitive to the size and balance of the labeled category dataset.

5.5 Zero-Shot BART (Dynamic Severity Classifier)

Goal: Provide real-time semantic classification of severity levels on arbitrary, previously unseen text inputs, without requiring a labeled training set for severity specifically.

Architecture: The model uses `facebook/bart-large-mnli`, a BART encoder-decoder fine-tuned on the Multi-Genre Natural Language Inference (MNLI) corpus, exposed through a Natural Language Inference (NLI) pipeline.

Zero-Shot Logic: Rather than training a dedicated severity classifier, the CVE description is treated as a “premise,” and each candidate severity label is converted into a natural-language “hypothesis,” for example “*This vulnerability is high severity.*” The model estimates the entailment probability between the premise and each hypothesis, and the label with the highest entailment score is selected as the predicted severity.

Advantage: Because no severity-specific fine-tuning is required, this component can be adapted to new or revised severity taxonomies (e.g. adding a “Critical-Plus” tier) simply by changing the hypothesis templates, without retraining any weights.

Trade-off: Zero-shot classification is generally less accurate than a model fine-tuned directly on labeled severity data, so its output is treated as a corroborating signal alongside the Random Forest’s regression-based severity prediction rather than as the sole source of truth.

5.6 Combined Pipeline Behavior

At inference time, a single CVE description is passed through all five models in parallel. The Random Forest and Zero-Shot BART independently estimate severity from two different representations (structured TF-IDF features versus zero-shot semantic entailment), the DistilBERT classifier assigns a specific vulnerability category, and the Gaussian HMM and MEMM jointly characterize how the current vulnerability fits into the broader temporal threat sequence. The aggregation layer then reconciles these five outputs – for example, flagging disagreement between the Random Forest and BART severity estimates for manual review, or using the HMM’s state sequence to contextualize a spike in DistilBERT-classified Remote Code Execution vulnerabilities – before writing the final, unified prediction to the SQLite vulnerability vault.

6 Implementation

6.1 Development Environment

- Programming Language: Python
- Backend Framework: FastAPI
- Machine Learning Libraries: Scikit-learn, PyTorch, Transformers
- Frontend: HTML5, CSS3, JavaScript
- Database: SQLite
- Development Environment: VS Code
- Version Control: Git & GitHub

6.2 Installation and Execution

1. Clone Repository

```
git clone https://github.com/Shivani367/NLP_CVE.git
```

2. Create Virtual Environment

```
py -3.11 -m venv venv
```

3. Activate Environment

```
venv\Scripts\activate
```

4. Install Dependencies

```
pip install -r requirements.txt
```

5. Run Application

```
python main.py
```

Application URL: <http://127.0.0.1:8000/>

6.3 Backend Implementation

The backend, built with FastAPI, performs the following operations:

- Loads trained models at startup
- Accepts CVE query input via API endpoints
- Executes preprocessing and feature extraction
- Runs prediction across all five models
- Aggregates outputs into unified threat intelligence
- Stores results in the SQLite vulnerability vault
- Returns JSON responses for the dashboard to render

6.4 System Integration

1. CVE query is submitted via the dashboard.
2. Data is sent to the FastAPI backend.
3. Preprocessing and feature extraction are performed.
4. All models generate predictions in parallel.
5. Predictions are aggregated and stored in SQLite.
6. Final output is returned and rendered on the dashboard.

This integration ensures seamless interaction between components and provides near real-time vulnerability intelligence.

7 Dashboard Modules and User Interface

The dashboard is organized into seven functional modules, each surfacing a different view of the underlying threat intelligence.

7.1 Vulnerability Center

Provides the CVE query analyzer, CVSS score prediction, severity classification, and ML prediction confidence.

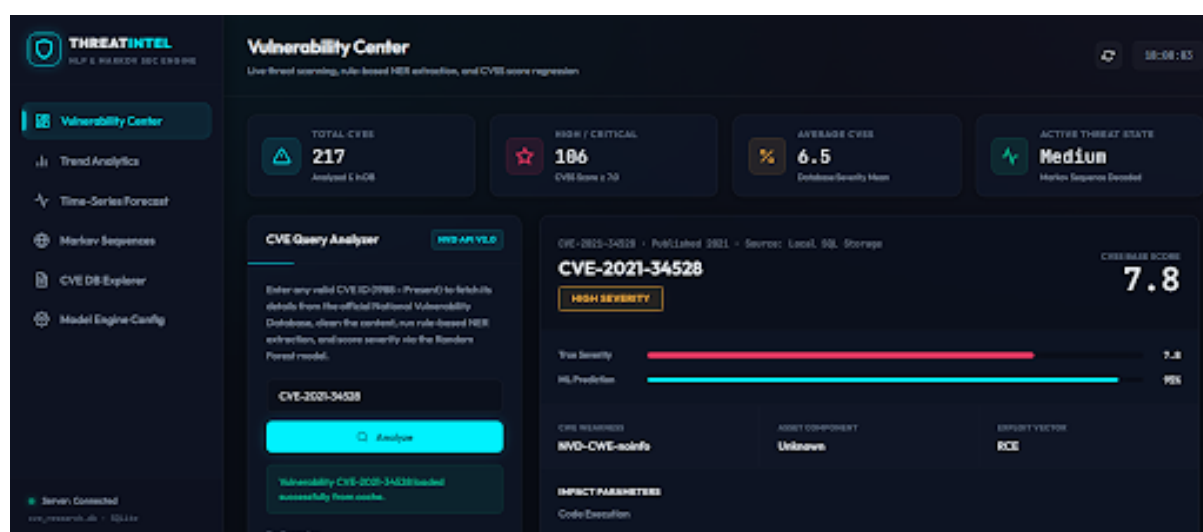


Figure 7.1: Web dashboard for manual email analysis and real-time prediction

Observed output on the current dataset: Total CVEs – 217; High/Critical – 106; Average CVSS – 6.5; Active Threat State – Medium.

7.2 Trend Analytics

Shows exploit vector breakdown, named-entity classification, top affected components, and historical threat distribution across categories such as RCE, SQL Injection, Buffer Overflow, XSS, and DoS.



Figure 7.2: Web dashboard for manual email analysis and real-time prediction

7.3 CVSS Distribution Dashboard

Displays CVSS density analysis, statistical visualization, and severity trend monitoring. Critical vulnerabilities in the current dataset are concentrated between CVSS ranges 7–9.



Figure 7.3: Web dashboard for manual email analysis and real-time prediction

7.4 Time-Series Forecast

Combines ARIMA forecasting and linear regression trends to compare historical vs. future vulnerability disclosure rates, forecasting from 2017 through 2028.



Figure 7.4: Web dashboard for manual email analysis and real-time prediction

7.5 Markov Threat Sequences

Visualizes Gaussian HMM threat-state clustering and chronological sequence analysis across Low, Medium, and High threat states.



Figure 7.5: Web dashboard for manual email analysis and real-time prediction

7.6 SQLite Vulnerability Vault

A searchable CVE database explorer with severity filtering and CSV export, storing CVE IDs, CVSS scores, exploit types, affected components, and severity levels.

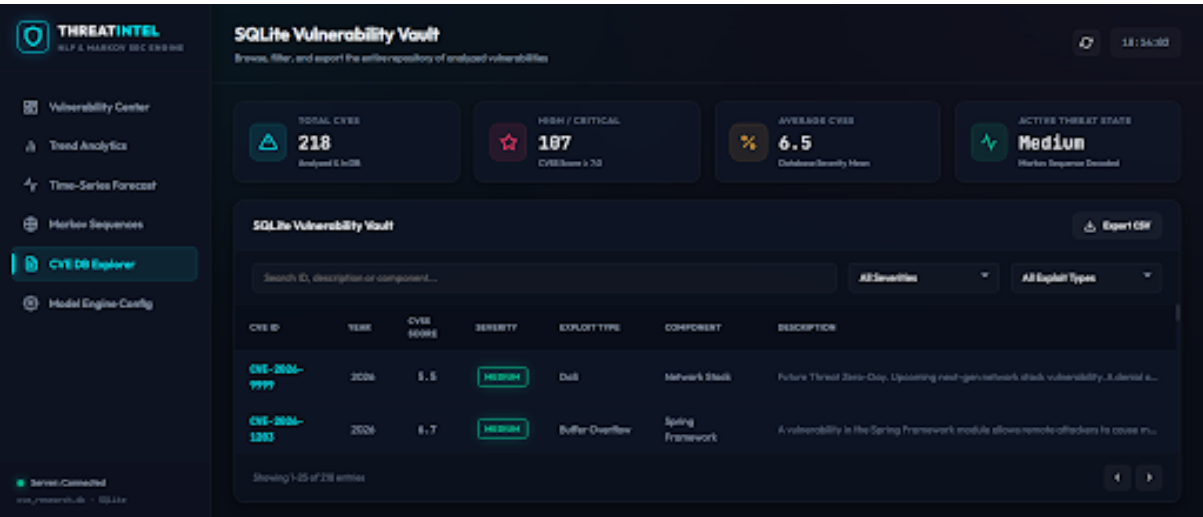


Figure 7.6: Web dashboard for manual email analysis and real-time prediction

7.7 Model Engine Configuration

Allows retraining of the Random Forest, HMM, and MEMM models, fine-tuning of Dis-tilBERT, database reset, and third-party API configuration.

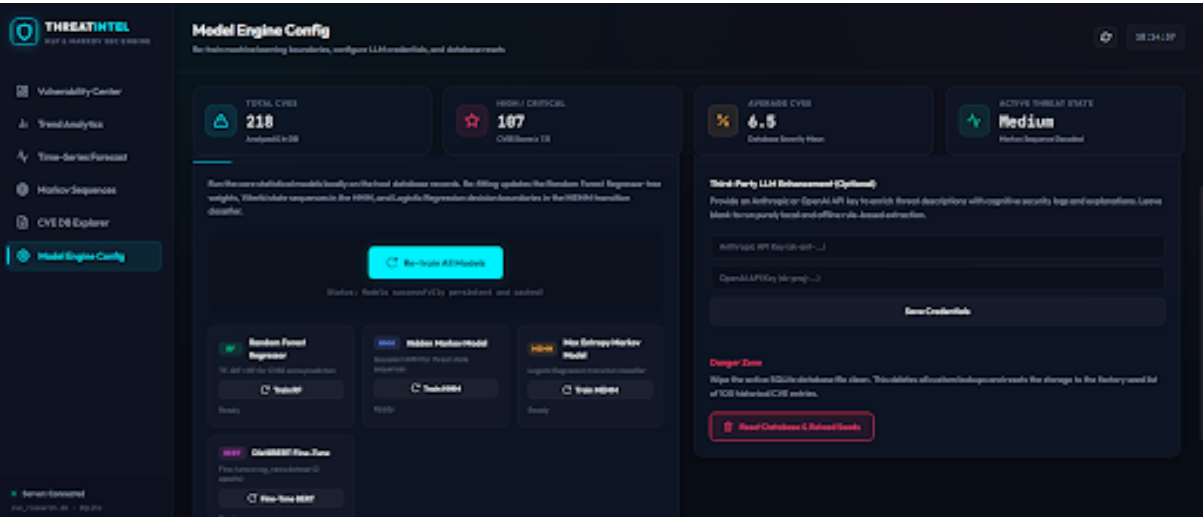


Figure 7.7: Web dashboard for manual email analysis and real-time prediction

8 Advantages and Limitations

8.1 Advantages

- Automated, end-to-end vulnerability analysis
- Fast severity prediction with interpretable confidence scores
- Real-time analytics and visualization
- Combines classical ML, sequence modeling, and transformer-based NLP for cross-validated predictions
- Interactive, scalable dashboard architecture
- Offline model support

8.2 Limitations

- High memory usage due to transformer model size
- GPU acceleration recommended for DistilBERT and BART inference
- Initial dependency installation time can be significant
- Effectiveness depends on the quality and balance of training data

9 Conclusion and Future Scope

9.1 Conclusion

This project presents the CVE Threat Intelligence Dashboard, a comprehensive cybersecurity analytics platform that integrates Natural Language Processing, classical machine learning, probabilistic sequence modeling, and transformer-based deep learning into a single, unified pipeline for vulnerability intelligence.

By combining a Random Forest regressor, Gaussian HMM, MEMM, fine-tuned DistilBERT, and zero-shot BART, the system produces cross-validated predictions of CVSS severity, exploit category, and threat-state transitions directly from raw CVE text. An SQLite-backed vulnerability vault and an interactive dashboard make the resulting intelligence queryable, exportable, and visually interpretable, while ARIMA-based forecasting extends the system’s utility from reactive triage to proactive trend monitoring.

The result is a scalable, enterprise-oriented tool that demonstrates how multiple AI techniques can be layered together to automate a task – CVE triage – that is traditionally slow and manual.

9.2 Future Scope

- **Cloud Deployment:** Hosting the platform on cloud infrastructure for wider accessibility.
- **SIEM Integration:** Connecting the dashboard to enterprise Security Information and Event Management systems.
- **Real-Time CVE Feeds:** Ingesting live CVE feeds instead of static datasets.
- **Threat Intelligence APIs:** Integrating external threat intelligence sources for cross-referencing.
- **Multi-User Authentication:** Supporting role-based access for security teams.
- **Kubernetes Deployment:** Containerized, horizontally scalable deployment.
- **LLM-Assisted Threat Explanations:** Using large language models to generate human-readable summaries of predicted threats.

In summary, the CVE Threat Intelligence Dashboard provides a scalable, accurate, and practical foundation for automated vulnerability analysis, with a clear path toward enterprise-grade deployment.